

Flutter と Dart のプロジェクト構造の紹介

◇ 概要：Flutter アプリは何で構成されているのか？

- Flutter は Dart を使用し,オブジェクト指向プログラミング (OOP) でモジュール化されている.
- Flutter アプリはウィジェットのコレクションで構成される.
- すべての UI 要素 (テキスト,画像,ボタン,レイアウト) はウィジェットである.
- Flutter アプリはウィジェット階層構造を持ち,ウィジェットは他のウィジェットの中にネストできる (親子関係) .

◇ Flutter アプリの基本構造 (コード例)

- Flutter アプリは `runApp()` で開始する.
- `MaterialApp` はアプリの主要な構造を定義する.
- 各 UI 要素 (`Text`, `AppBar`, `Scaffold`) はウィジェットである.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Flutter App'),
        ),
        body: Center(
          child: Text('Hello, Flutter!'),
        ),
      ),
    );
  }
}
```

▪ 説明：

- `runApp(MyApp())` → アプリを起動する.
- `MaterialApp` → アプリ全体の構造を定義する.
- `Scaffold` → アプリのレイアウト (`AppBar`, `body` など) を提供する.
- `AppBar` → トップナビゲーションバーを定義する.
- `Text('Hello, Flutter!')` → アプリ内にテキストを表示する.

構造の分解（ステップごとの解説）

ステップ1：インポート 'package:flutter/material.dart' の説明

a. プログラミングにおけるインポートとは？

- プログラミングにおける「インポート」とは、他のファイルやライブラリからコードを取り込み、プロジェクトで使用することを意味する。
- 工具箱から必要なツールを借りて作業をするのと同じようなもの。
- Flutter では、事前に用意されたウィジェットや関数を使うためにライブラリのインポートが必要。

b. import 'package:flutter/material.dart'; はどういう意味か？

- これは、Flutter の UI コンポーネントを含むライブラリ。
- flutter → Flutter フレームワークのライブラリであることを示す。
- material → Google の「マテリアルデザイン」スタイルを使うことを意味する。
- dart → Dart プログラミング言語で書かれているライブラリ。
- この1行で、Flutter に用意された多くの UI コンポーネントにアクセスできるようになる。

```
import 'package:flutter/material.dart';
```

c. なぜこれを使うのか？

- ボタン、テキスト、レイアウトなどの UI コンポーネントを使用できるようにする。
- このインポートがなければ、Flutter の基本ウィジェットを使えない。
- 視覚的に魅力的なアプリを簡単に構築できるようになる。

d. material.dart によって提供される主要なウィジェット

ウィジェット	使用目的	例
Text	画面にテキストを表示する	<code>Text('Hello, Flutter!')</code>
ElevatedButton	インタラクティブなボタンを作成する	<code>ElevatedButton(onPressed: () {}, child: Text('Click Me'))</code>
Scaffold	基本的なレイアウト構造を提供する	<code>Scaffold(appBar: AppBar(title: Text('App')), body: Center(child: Text('Hello')))</code>
AppBar	上部のナビゲーションバーを作成する	<code>AppBar(title: Text('My App'))</code>

- これらのウィジェットはアプリの構造を作り、ユーザーとのインタラクションを定義する。
- `import 'package:flutter/material.dart';` がないと、これらの UI 要素を使用できない。

ステップ2：Flutter の main.dart の基本を理解する

a. Flutter における main() の役割

- Flutter アプリは main() から開始される (Java と同様) .
- ただし,main() の後に runApp() を呼び出す必要がある.
- runApp(MyApp()) は MyApp ウィジェットを表示し,UI を初期化する.
- これは,main.dart における基本的な Flutter コードの例:

```
void main() {  
  runApp(MyApp());  
}
```

- 説明:
 - ◇ main() → プログラムの開始点.
 - ◇ runApp(MyApp()) → Flutter アプリを起動し,MyApp() をロードする.

b. Flutter における runApp() の仕組み

- runApp() はアプリのルートウィジェットをロードする.
- MyApp() ウィジェットを表示し,UI 階層を構築する.
- Android の XML のような UI ファイルを使わず,すべてウィジェットで構築される.

c. 比較: Java の main() と Flutter の main()

- Java と Flutter はどちらも main() から実行が開始されるが,UI の扱いが異なる.
- Java (例: Android アプリ) → main() はロジックを開始するだけで,UI は XML で別途構築.

```
public class Main {  
  public static void main(String[] args) {  
    System.out.println("Hello World");  
  }  
}
```

- Flutter/Dart → main() の後,すぐにウィジェットで UI を構築.
- 主な違い:

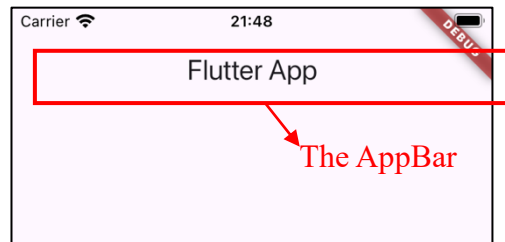
機能	Java (Android アプリ)	Flutter (Dart)
UI の処理	UI は XML で別途作成	UI は main.dart で直接構築
ワークフロー	main() でアプリを起動し, UI は別途処理	main() でアプリと UI を同時にセットアップ
管理のしやすさ	UI とロジックが分かれている	すべて 1 つのファイルで管理

ステップ3：MyAppとウィジェットツリーを理解する

a. MyAppとは何か？

- `runApp()` を呼び出した後、アプリは **MyApp** ウィジェットをロードする。
- **MyApp** は Flutter アプリのルートウィジェット。
- すべての Flutter アプリはウィジェットで構成され、UI を形成する。
- これは基本的な **MyApp** の実装例：

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(  
          title: Text('My Flutter App'),  
        ),  
        body: Center(  
          child: Text('Hello, world!'),  
        ),  
      ),  
    );  
  }  
}
```



- 説明:
 - ◇ **MyApp** → アプリのエントリーポイントとして機能するカスタムウィジェット。
 - ◇ `StatelessWidget` → **MyApp** は状態を持たない（ステートレス）。
 - ◇ `build()` メソッド → UI の表示方法を定義する。
 - ◇ ウィジェットの階層構造（ウィジェットツリー）を返す。

b. ウィジェットツリーの仕組み

- Flutter の UI はツリー構造（ウィジェットツリー）で作られている。
- 各ウィジェットは子ウィジェットを持つことができ、階層を形成する。
- ウィジェットツリーはウィジェット同士の構造と関係性を定義する。
- これは **MyApp** 内のウィジェットツリーの例：

```
MaterialApp(  
  home: Scaffold(  

```

```

    appBar: AppBar(
      title: Text('My Flutter App'),
    ),
    body: Center(
      child: Text('Hello, world!'),
    ),
  ),
);

```

■ ツリー構造：

```

MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text ('My Flutter App')
│   └── Body
│       └── Center
│           └── Text ('Hello, world!')

```

■ 説明:

- ◇ MaterialApp → すべてを包むルートウィジェット。
- ◇ Scaffold → アプリの基本構造を提供する（AppBar, body など）。
- ◇ AppBar → アプリ上部のバー（タイトルやアクション用）。
- ◇ Text → 画面にテキストを表示するウィジェット。

c. ツリー内の各ウィジェットの理解

ウィジェット	使用目的	例
MaterialApp	テーマやナビゲーションを設定するルートウィジェット	MaterialApp(home: Scaffold(...))
Scaffold	画面の基本レイアウト（AppBar や body など）を提供する	Scaffold(appBar: ..., body: ...)
AppBar	上部ナビゲーションバーを表示する	AppBar(title: Text('My App'))
Center	子ウィジェットを画面の中央に配置する	Center(child: Text('Hello'))
Text	シンプルなテキストを表示する	Text('Hello, world!')

- Flutter の各ウィジェットは特定の機能を持ち,他のウィジェットの中にネストできる。
- このネスト構造により,UI を効率的に整理できる。

ステップ4：クラスとエクステンドを理解する

a. Flutterにおけるクラスとは？

- クラスはオブジェクト（Flutterではウィジェット）を作るための設計図.
- Flutterアプリは,クラスとして定義されたカスタムウィジェットで構築される.
- 各ウィジェットクラスは,UIの外観と動作を定義する.
- Flutterでウィジェットクラスを定義する一般的な方法はこちら：

```
class MyApp extends StatelessWidget {
```

▪ 説明:

- ◇ `class MyApp` → `MyApp` という名前のウィジェットを定義する.
- ◇ `extends StatelessWidget` → `MyApp` は `StatelessWidget` を継承しており,動的に変化しないウィジェットである.

b. Flutterにおける「extends」の意味とは？

- `extends` は,あるクラスを継承し,新しいクラスを作るためのキーワード.
- `extends StatelessWidget` を使うと,ウィジェットが変更されない（ステートレス）.
- ウィジェットは,親ウィジェットがリビルドしない限り更新されない.
- これは `Stateless` ウィジェットの完全な例：

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(title: Text('My Flutter App')),  
        body: Center(child: Text('Hello, Flutter!')),  
      ),  
    );  
  }  
}
```

▪ 説明:

- ◇ `class MyApp extends StatelessWidget` → `MyApp` はステートを持たないウィジェット.
- ◇ `build()` メソッド → ウィジェットのUIを定義する.
- ◇ `MaterialApp` → アプリのルートウィジェット.

c. `super.key` とは何か,なぜ使うのか？

- `super.key` は,`StatelessWidget` や `StatefulWidget` などの親クラスに `key` を渡すために使われる.
- Flutterがウィジェットツリー内のウィジェットを効率的に識別・更新するのに役立つ.

- 必須ではありませんが,ウィジェットの最適化や再利用性を高めるために 推奨される.

d. Java との比較

- Dart も Java も `extends` を使ってサブクラスを作る.
- Dartの方がシンプルで,Flutterのウィジェット設計に最適化されている.
- Javaでは通常,UIは別ファイルで定義されるが,Flutterではウィジェットクラス内に直接書く.

```
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

ステップ5 : `@override` を使ったメソッドのオーバーライド

a. Flutterにおける`@override`とは？

- `@override` は,子クラスのメソッドが親クラスのメソッドを置き換えるときに使う.
- Flutterでは,`@override` は `StatelessWidget` や `StatefulWidget` から継承された `build()` によく使用される.
- 親クラスのメソッドを正しくオーバーライドしていることを保証する.
- `build()`メソッドをオーバーライドする例：

```
@override
Widget build(BuildContext context) {
```

- 説明:
 - ◇ `@override` → この `build()` メソッドが親クラスの `build()` を置き換えていることをFlutterに伝える.
 - ◇ `build(BuildContext context)` → UIの表示方法を定義する.
 - ◇ `@override` がなくても Dartは動作するが,コードの明確性を向上させ,ミスを防ぐのに役立つ.

b. なぜ`@override`が重要なのか？

- メソッドが正しくオーバーライドされていることを保証する.
- コードを読みやすくする → このメソッドが親クラスのメソッドを置き換えていることを明示的に示す.
- 意図しないミスを防ぐ → `@override` がないと,メソッド名を間違えたときに Dartがエラーを検出しない可能性がある.

c. 比較：Java と Flutter における`@override`

- Java も Flutter (Dart) も `@override` を使って,メソッドのオーバーライドを明示する.

- メソッドのシグネチャが一致しない場合のミスを防ぐ.
- Java では明示的なメソッドシグネチャが必要だが,Dart はより柔軟.

```
class Parent {
    void build() {
        System.out.println("Parent build method");
    }
}

class Child extends Parent {
    @Override
    void build() { // Overriding the parent method
        System.out.println("Child build method");
    }
}
```

- 説明:
 - ◇ `@Override` → Child の `build()` が Parent の `build()` を正しくオーバーライドしていることを保証する.
 - ◇ メソッド名が間違っていた場合,Java はエラーを出す.
- 主な違い:

機能	Java	Flutter
<code>@override</code> の用途	OOP メソッドのオーバーライド	Flutter ウィジェットのオーバーライド
エラー防止	誤ったメソッドシグネチャを防ぐ	タイプミスの検出を助ける
柔軟性	厳格なメソッド定義	より柔軟だが, <code>@override</code> で明確

ステップ 6 : `build()` メソッドと `BuildContext` の探索

a. `build()` メソッドとは？

- `build()` メソッドは,ウィジェットが画面にどのように表示されるかを定義する.
- “ウィジェットツリー” (UI を構成するウィジェットの階層構造) を返す.
- UI が更新される必要があるときに,`build()` メソッドが再び呼び出される.

```
class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                body: Center(
                    child: Text('Hello, Flutter!'),
                ),
            ),
        );
    }
}
```



```
}
```

- 説明:
 - ◇ build() は MaterialApp ウィジェットを返し,アプリの UI 全体を定義する.
 - ◇ MaterialApp の中に Scaffold,Center,Text などのウィジェットが含まれる.
 - ◇ この構造がウィジェットツリーを形成し,UI のレイアウトを決める.

b. BuildContext とは？

- BuildContext は,ウィジェットがウィジェットツリーのどこにあるかを示す情報を提供する.
- 継承されたウィジェットにアクセスしたり,ナビゲーションを管理したりできる.
- build() メソッドの必須パラメータである.

c. build()が UI を更新し続ける仕組み

- Flutter は UI の変化を検出すると build() を再び呼び出す.
- UI の最新状態を確実に表示するために build() を利用する.
- Flutter は宣言的 UI モデルを採用しており,手動で UI を変更するのではなく,再構築する.

d. 比較：Java における UI の構築方法との違い

- Java (Android) では,通常,UI の更新は View オブジェクトの変更によって行われる.
- Flutter では,UI は宣言的に構築され,build() を使って再構築される.
- Flutter のアプローチにより,UI の更新が構造化され,予測しやすくなる.

```
@Override
protected View build(Context context) {
    TextView textView = new TextView(context);
    textView.setText("Hello, Android!");
    return textView;
}
```

- 主な違い

機能	Java (Android)	Flutter (Dart)
UI の構造	XML や Java コードで View を作成	build() メソッドでウィジェットツリーを返す
状態管理	View を手動で更新	状態が変わると UI が自動更新
柔軟性	View 階層を変更する必要がある	build() によりシンプルに管理

ステップ 7：Flutter における return の役割

a. build()における return の役割とは？

- build() メソッド内で return を使って,ウィジェットが表示する内容を指定する.

- レンダリングのためにウィジェットを Flutter に送り返す.
- Flutter は return を使って UI 要素を構築し,動的に表示する.

b. return で返されるウィジェットの理解

ウィジェット	使用目的	例
MaterialApp	ナビゲーション, テーマ, ローカライゼーションを提供するルートウィジェット	<code>return MaterialApp(...)</code>
Scaffold	AppBar や body などを含むメイン画面の構造を定義する	<code>return Scaffold(...)</code>
AppBar	アプリのタイトルや操作を表示する上部ナビゲーションバー	<code>AppBar(title: Text('My App'))</code>
Text	画面にシンプルなテキストを表示する	<code>Text('Hello, world!')</code>

- ポイント:
 - ◇ Flutter では,build() 内で return を使ってウィジェットを返さないと画面に表示されない.
 - ◇ return がないと,Flutter は何を表示すべきか判断できない.

c. 比較: Java における return の使われ方

- Java も Flutter (Dart) も,return を使って作成したオブジェクトを返す.
- Java では return は View や UI コンポーネントを返すことが多い.
- Flutter では,return はよりシンプルでウィジェット中心の設計になっている.
- 主な違い:

機能	Java (Android)	Flutter (Dart)
戻り値の型	View (TextView,Button など) を返す	Widget (Text,Scaffold など) を返す
UI の構造	UI は通常 XML で管理	Dart 内でウィジェットを使って UI を構築
柔軟性	View の階層を変更する必要がある	build() メソッド内で簡単に管理可能

ステップ 8 : Flutter の MaterialApp とは ?

a. MaterialApp とは ?

- MaterialApp は,Google の Material Design に準拠した Flutter アプリの基盤となる.
- アプリの基本構造とビジュアルスタイルを提供する.
- テーマ,ナビゲーション,デフォルトの動作を設定し,スムーズなユーザー体験を実現する.